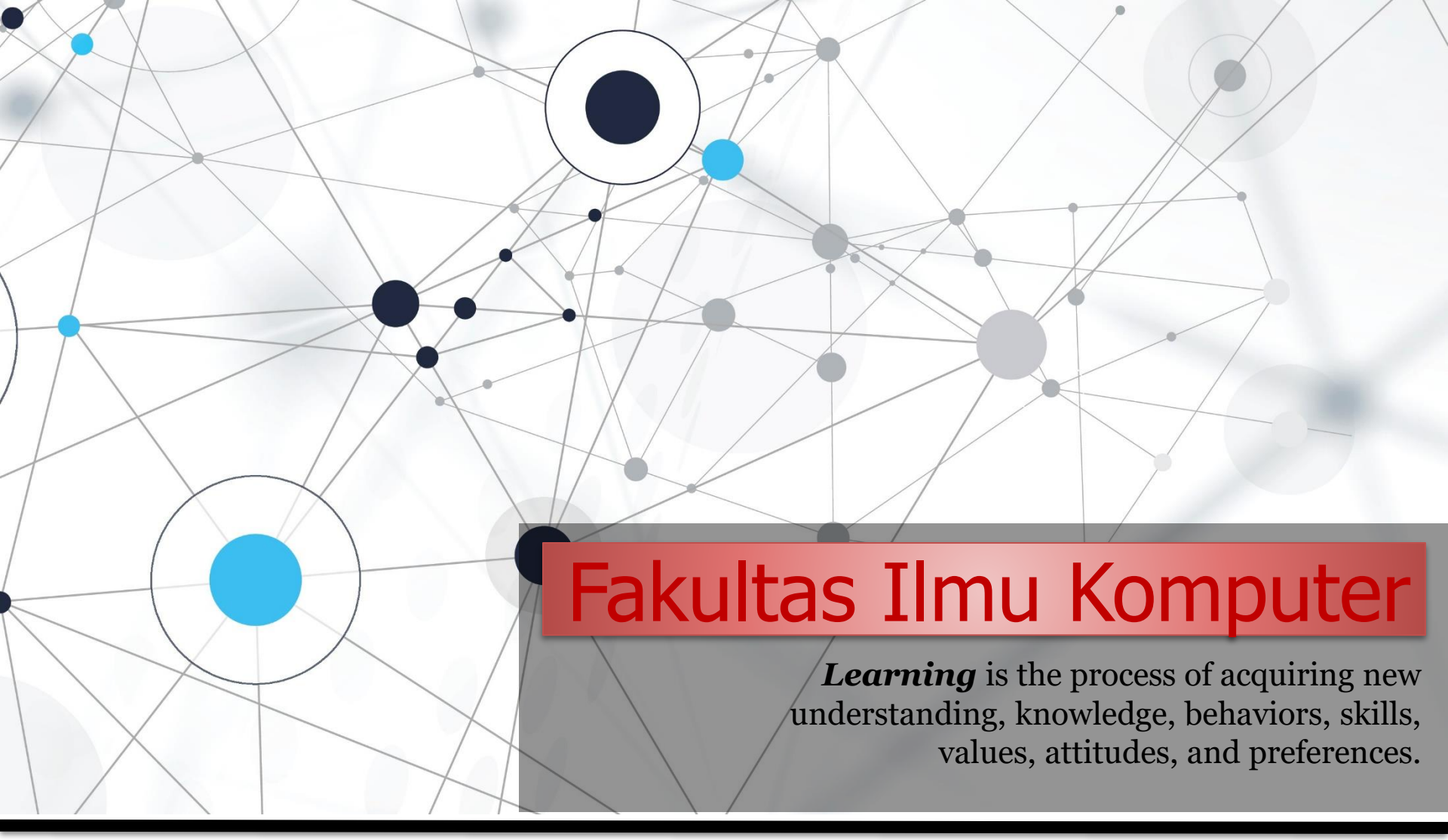
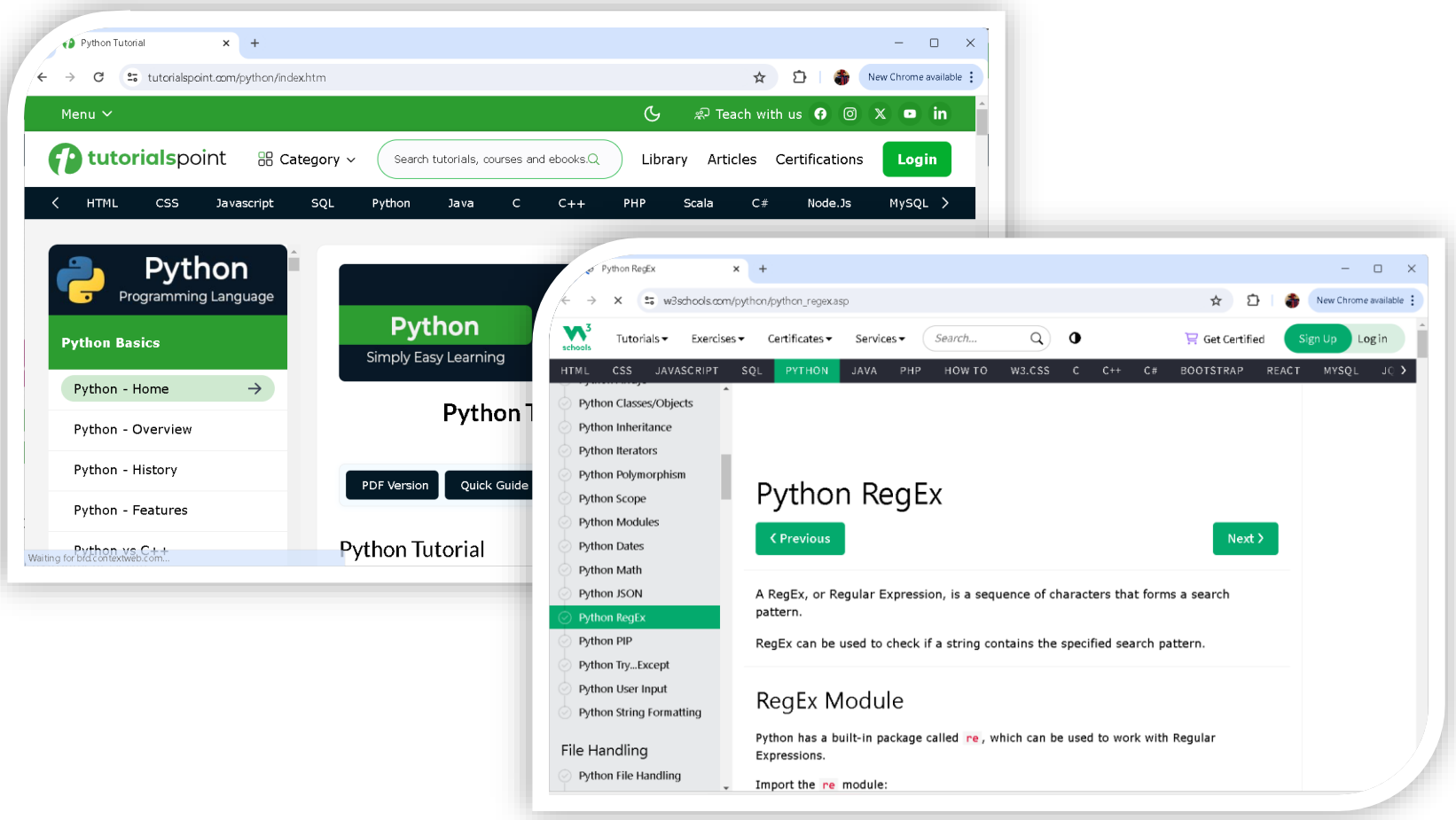


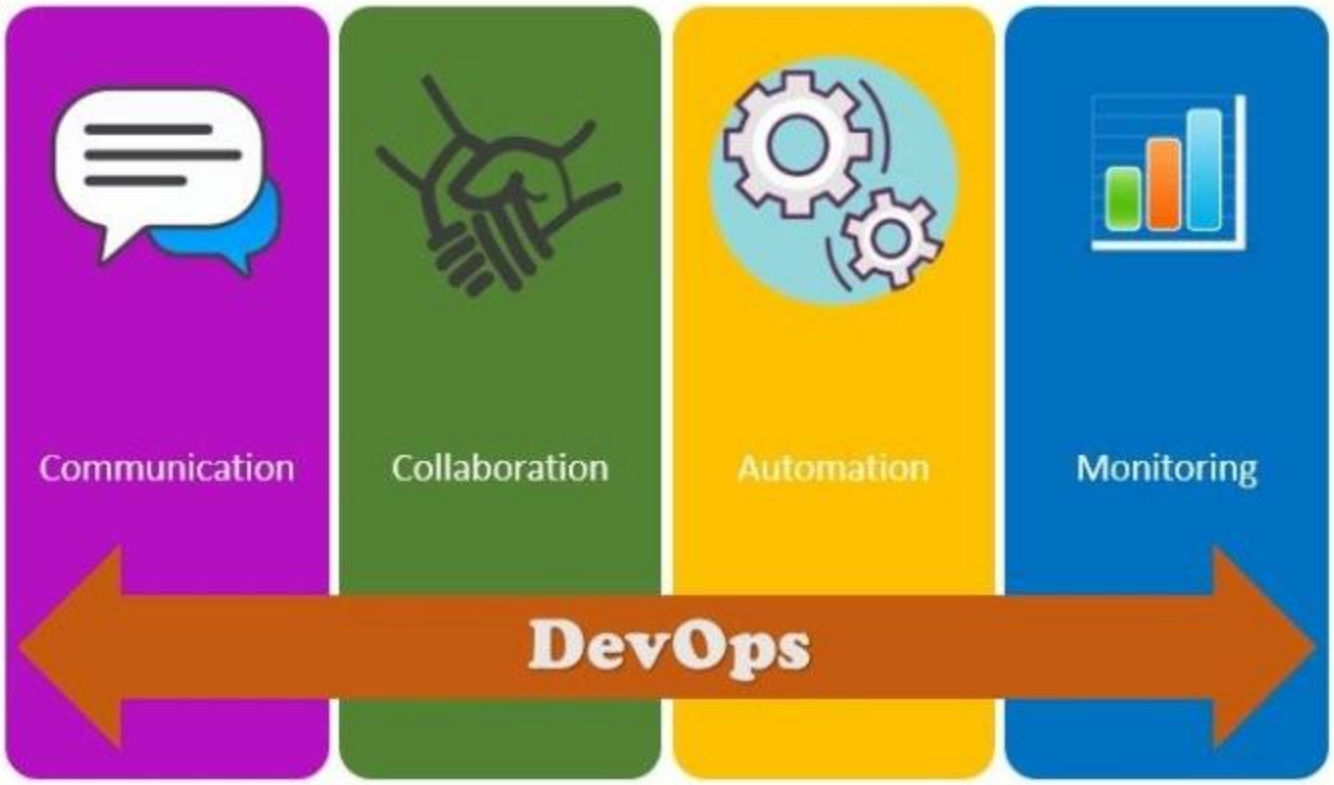
OOP in Python – Files and Strings
OOP in Python – Exception and Exception Handling

Welcome to *the* Object Oriented Programming *course*



Sumber materi diambil dari **tutorialpoint** & **W3School** !!!

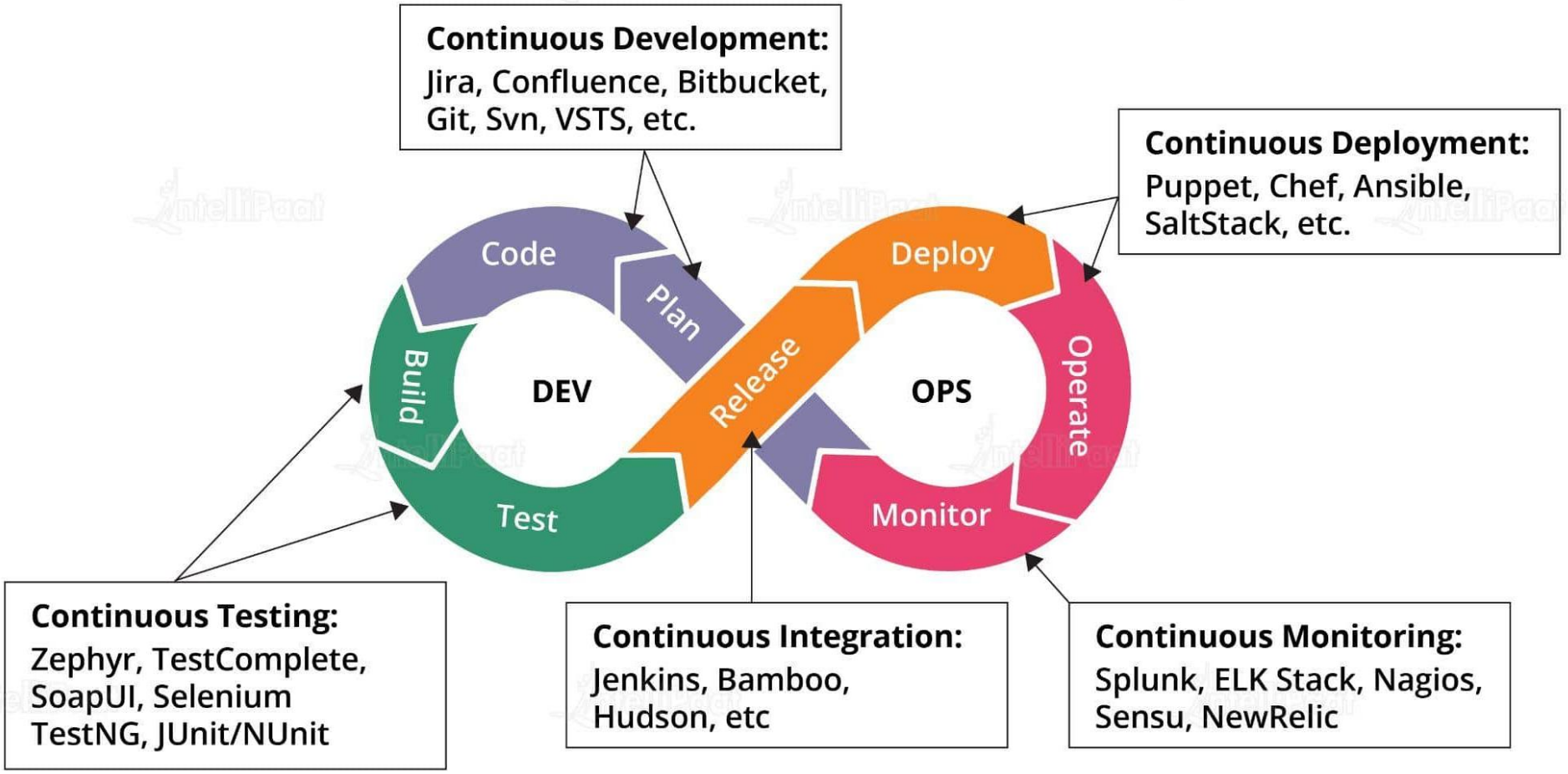
The 4 Pillars *of* DevOps

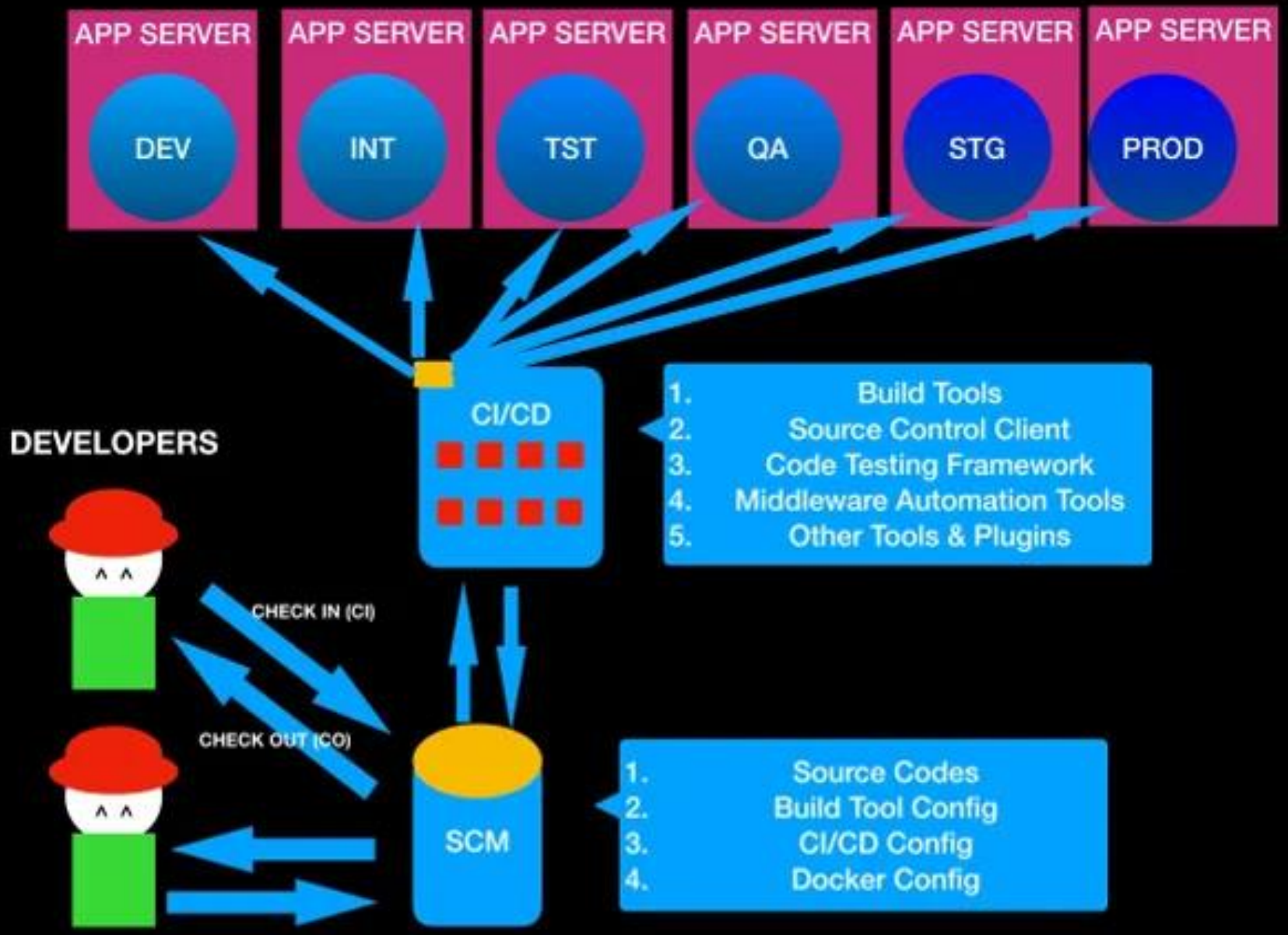


DevOps is a movement focused on improving **communication**, **collaboration** and **integration** between the software development and IT operation teams. DevOps emphasizes on **BRIDGING THE GAP** between these teams by focusing on delivering software faster and creating a success metric.

Collaboration *in* DevOps

Online collaboration tools are software that lets **teams work together over the internet**. Examples include **Trello**, **Slack** and **Asana**. The purpose of these tools is **to facilitate communication**, **collaboration** and **teamwork** for remote teams.




APP SERVER
DEV
APP SERVER
INT
APP SERVER
TST
APP SERVER
QA
APP SERVER
STG
APP SERVER
PROD
CI/CD
1. Build Tools
2. Source Control Client
3. Code Testing Framework
4. Middleware Automation Tools
5. Other Tools & Plugins
DEVELOPERS
CHECK IN (CI)
CHECK OUT (CO)
SCM
1. Source Codes
2. Build Tool Config
3. CI/CD Config
4. Docker Config

2 - OOP in Python –Strings

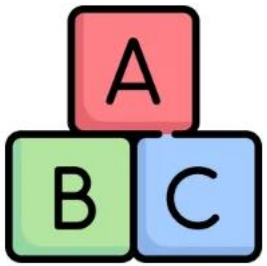
Strings

- **Strings are the most popular data types** used in every programming language.
- Why? Because we, understand text better than numbers, so in writing and talking we use **TEXT** and **WORDS**, similarly in programming too we use strings.
- In string we parse text, analyze text semantics, and do data mining – and all this data is human consumed text.
- The string in Python is IMMUTABLE. Python strings are "*immutable*" which means they cannot be changed after they are created (Java strings also use this immutable style).
- Strings in Python are arrays of bytes representing Unicode characters. A string is a collection of one or more characters put in a single quote, double-quote, or triple-quote. In Python, there is no character data type Python, a character is a string of length one. It is represented by str class.

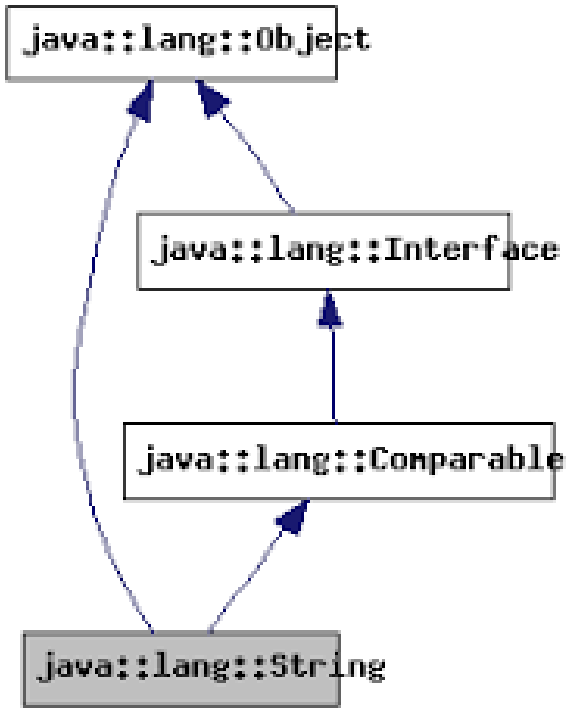
S = 'PrepInsta'

Indexing :

p	r	e	p	l	n	s	t	a
0	1	2	3	4	5	6	7	8
-9	-8	-7	-6	-5	-4	-3	-2	-1



String class *in* Java



The String class represents character strings. All string literals in Java programs, such as "abc" , are implemented as instances of this class. Java String class with methods such as concat, compareTo, split, join, replace, trim, length, intern, equals, comparison, substring operation.

Java String Methods

(1) length()
(2) charAt()
(3) trim()
(4) indexOf()
(5) lastIndexOf()
(6) replace()
(7) toLowerCase()
(8) toUpperCase()
(9) split()
(10) substring()
(11) equals()
(12) getBytes()
(13) concat()
(14) contains()
(15) compareTo
(16) startsWith()
(17) endsWith()
:
:
:
etc.

Python Data Types

Name	Type	Description
Integers	int	Whole numbers, such as: 3 300 200
Floating point	float	Numbers with a decimal point: 2.3 4.6 100.0
Strings	str	Ordered sequence of characters: "hello" 'Sammy' "2000" "楽しい"
Lists	list	Ordered sequence of objects: [10,"hello",200.3]
Dictionaries	dict	Unordered Key:Value pairs: {"mykey" : "value" , "name" : "Frankie"}
Tuples	tup	Ordered immutable sequence of objects: (10,"hello",200.3)
Sets	set	Unordered collection of unique objects: {"a","b"}
Booleans	bool	Logical value indicating True or False

String Manipulation

- In Python, string can be marked in multiple ways, using single quote (‘), double quote(“) or even triple quote (‘”) in case of MULTILINE STRINGS.

- String manipulation is very useful and very widely used in every language. Often, programmers are required to break down strings and examine them closely.

- Strings can be iterated over (*character by character*), sliced, or concatenated. The syntax is the same as for lists.

- The **str class** has numerous methods on it to make manipulating strings easier. The *dir* and help commands provides guidance in the Python interpreter how to use them.

```
>>> # String Examples
>>> a = "hello"
>>> b = ''' A Multi line string,
Simple!'''
>>> e = ('Multiple' 'strings' 'togethers')
```

Common String Methods

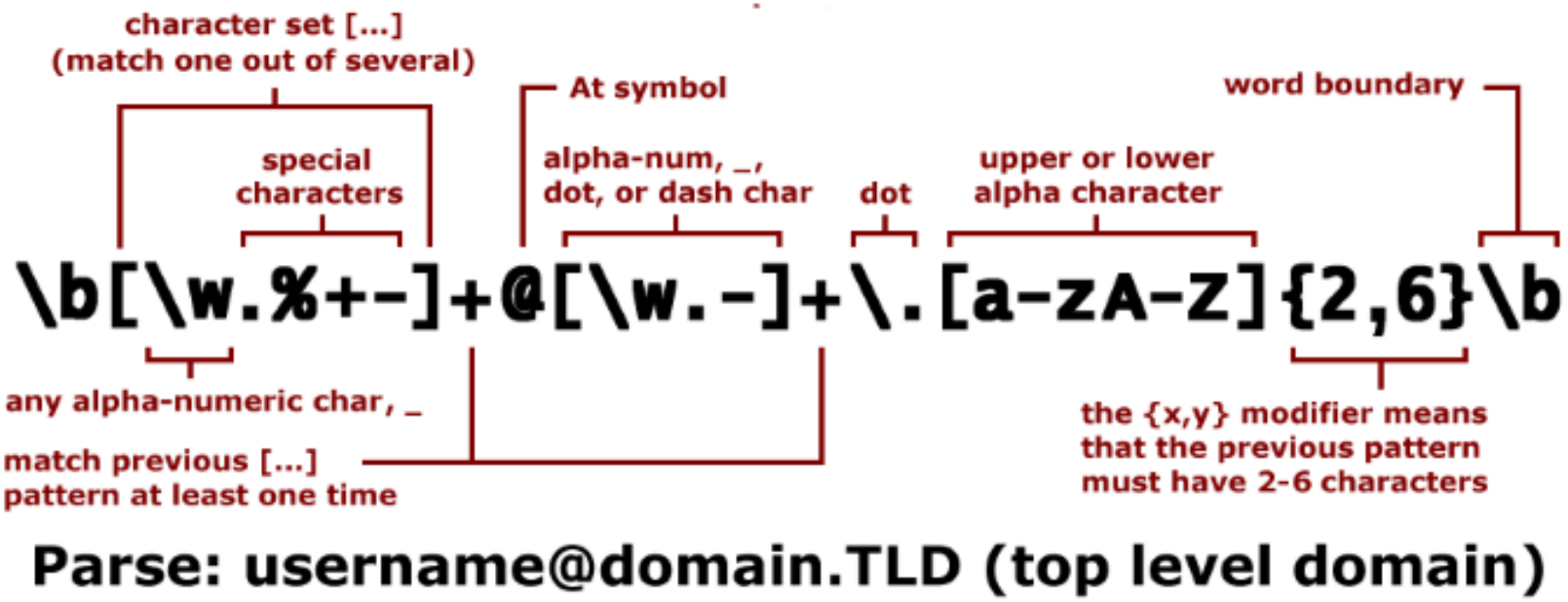
Sr.No.	Method & Description
1	**isalpha()** Checks if all characters are Alphabets
2	**isdigit()** Checks Digit Characters
3	**isdecimal()** Checks decimal Characters
4	**isnumeric()** checks Numeric Characters
5	**find()** Returns the Highest Index of substrings
6	**istitle()** Checks for Titlecased strings
7	**join()** Returns a concatenated string
8	**lower()** returns lower cased string
9	**upper()** returns upper cased string
10	**partion()** Returns a tuple
11	**bytearray()** Returns array of given byte size
12	**enumerate()** Returns an enumerate object
13	**isprintable()** Checks printable character

How to run string methods?

```
>>> str1 = 'Hello World!'
>>> str1.startswith('h')
False
>>> str1.startswith('H')
True
>>> str1.endswith('d')
False
>>> str1.endswith('d!')
True
>>> str1.find('o')
4
>>> #Above returns the index of the first occurence of the character/substring.
>>> str1.find('lo')
3
>>> str1.upper()
'HELLO WORLD!'
>>> str1.lower()
'hello world!'
>>> str1.index('b')
Traceback (most recent call last):
   File "<pyshell#19>", line 1, in <module>
      str1.index('b')
ValueError: substring not found
>>> s = ('hello How Are You')
>>> s.split(' ')
['hello', 'How', 'Are', 'You']
>>> s1 = s.split(' ')
>>> '*'.join(s1)
'hello*How*Are*You'
>>> s.partition(' ')
('hello', ' ', 'How Are You')
>>>
```

Python RegEx

- A RegEx, or **Regular Expression**, is a sequence of characters that forms a search pattern.
- RegEx can be used to check if a string contains the specified search pattern.
- Python has a **built-in package** called *re*, which can be used to work with Regular Expressions.
  > *import reg*



Example Python Regex

Check if the string starts with "Saya" and ends with "Unklab“.

```python
# import modul
import re

# input text
txt = "Saya ada mahasiswa di Unklab"

# create pattern regex
x = re.search("^Saya.*Unklab$", txt)

# create condition
if x:
  print("YES, kalimat diinputkan cocok !!!")
else:
  print("Tidak cocok !!!")
```

```
YES, kalimat diinputkan cocok !!!
```

Kode diatas merupakan contoh penggunaan modul re (*regular expression*) dalam bahasa pemrograman Python untuk melakukan pencocokan pola pada sebuah teks.

RegEx Functions

The *re module* offers a set of functions that allows us to search a string for a match:

Function	Description
findall	Returns a list containing all matches
search	Returns a Match object if there is a match anywhere in the string
split	Returns a list where the string has been split at each match
sub	Replaces one or many matches with a string

Regex Metacharacters

Metacharacters are characters with a special meaning.

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
$	Ends with	"planet$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{}	Exactly the specified number of occurrences	"he.{2}o"
|	Either or	"falls|stays"
()	Capture and group	

Regex Sets

A set is a set of characters inside a pair of square brackets [] with a special meaning.

Set	Description
[arn]	Returns a match where one of the specified characters (`a` , `r` , or `n`) is present
[a-n]	Returns a match for any lower case character, alphabetically between `a` and `n`
[^arn]	Returns a match for any character EXCEPT `a` , `r` , and `n`
[0123]	Returns a match where any of the specified digits (`0` , `1` , `2` , or `3`) are present
[0-9]	Returns a match for any digit between `0` and `9`
[0-5][0-9]	Returns a match for any two-digit numbers from `00` and `59`
[a-zA-Z]	Returns a match for any character alphabetically between `a` and `z` , lower case OR upper case
[+]	In sets, `+` , `*` , `.` , `|` , `()` , `$` , `{}` has no special meaning, so `[+]` means: return a match for any `+` character in the string

The findall() Function

Return a list containing every occurrence of "ai“.

```
# import module
import re

# input text
txt = "The rain in Spain."

# create regex pattern
x = re.findall("ai", txt)

# print output
print(x)
```

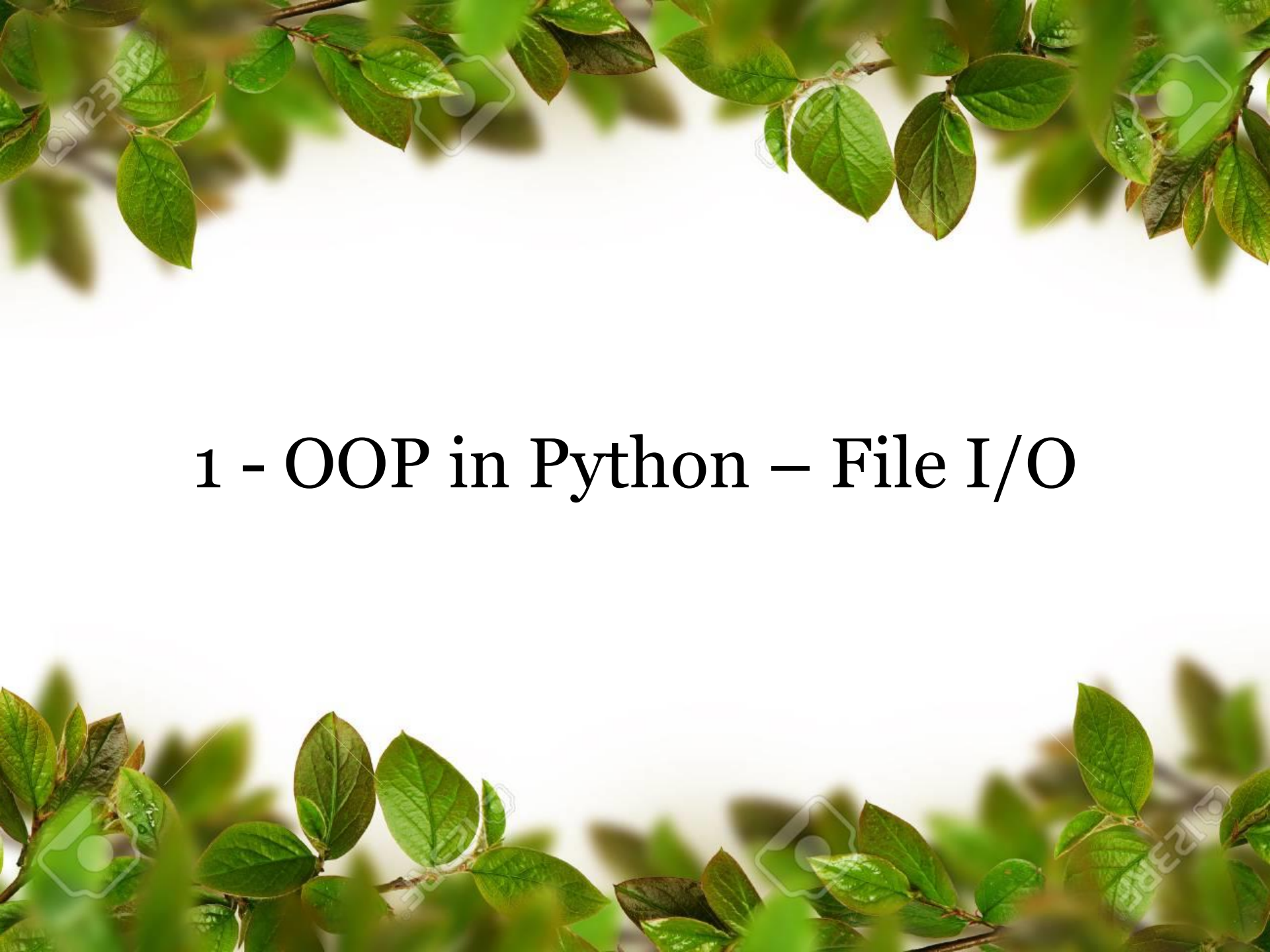
```
['ai', 'ai']
```

The search() Function

The *search() function* searches the string for a match, and returns a Match object if there is a match. If there is more than one match, only the first occurrence of the match will be returned.

```
# import package
import re

# input text
txt = "Saya mau makan nasi kuning."

# regex pattern
x = re.search("makan", txt)

# find position
print("Word 'makan' is located in position: ", x.start())
```

```
Word 'makan' is located in position:  9
```

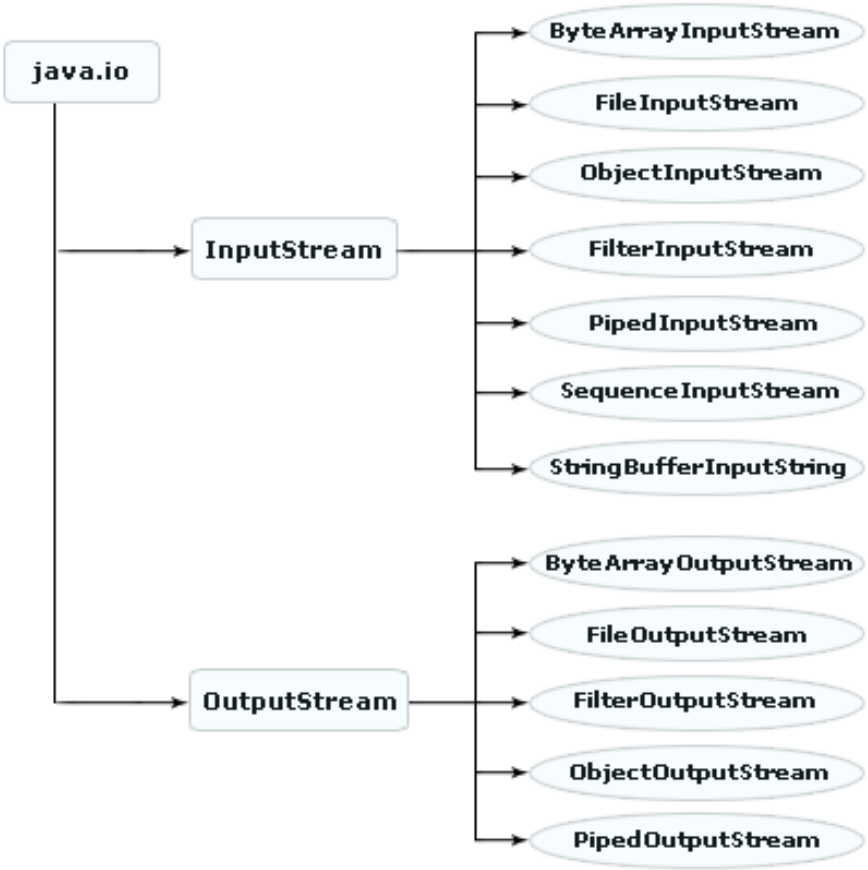


1 - OOP in Python – File I/O

File I/O

- File handling is an important part of any **WEB APPLICATION**.
- Python has several functions for creating, reading, updating, and deleting files.
- Operating systems represents files as a sequence of bytes, not text.
- A file is a named location on disk to store related information. It is used to permanently store data in your disk.
- In Python, a file operation takes place in the following order.
  - Open a file
  - Read or write onto a file (operation).
  - Close the file.

Java.io.File Class *in* Java



The File class from the **java.io package**, allows us to work with files. The File class has many useful methods for creating and getting information about files.

Method	Type	Description
canRead()	Boolean	It tests whether the file is readable or not
canWrite()	Boolean	It tests whether the file is writable or not
createNewFile()	Boolean	This method creates an empty file
delete()	Boolean	Deletes a file
exists()	Boolean	It tests whether the file exists
getName()	String	Returns the name of the file
getAbsolutePath()	String	Returns the absolute pathname of the file
length()	Long	Returns the size of the file in bytes
list()	String[]	Returns an array of the files in the directory
mkdir()	Boolean	Creates a directory

File Handling *in* Python

- The key function for working with files in Python is the open() function.

- The *open() function* takes two parameters; filename, and mode.

- There are four different methods (modes) for opening a file:
  - "*r*" - **Read** - Default value. Opens a file for reading, error if the file does not exist.
  - "*a*" - **Append** - Opens a file for appending, creates the file if it does not exist.
  - "*w*" - **Write** - Opens a file for writing, creates the file if it does not exist.
  - "*x*" - **Create** - Creates the specified file, returns an error if the file exists.
  - "*t*" - **Text** - Default value. Text mode
  - "*b*" - **Binary** - Binary mode (e.g. images)

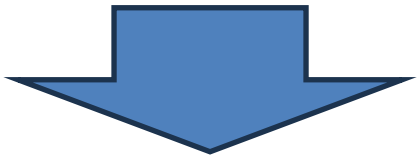
```
f = open("demofile.txt", "rt")
```

Because `"r"` for read, and `"t"` for text are the default values, you do not need to specify them.

Open File *in* Server

demofile.txt

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```



Example

```
f = open("demofile.txt", "r")
print(f.read())
```

Example

Open a file on a different location:

```
f = open("D:\\myfiles\welcome.txt", "r")
print(f.read())
```

Write *to* Existing File

To write to an existing file, we must add a parameter to the *open() function*:

- "**a**" - **Append** - will append to the end of the file.
- "**w**" - **Write** - will overwrite any existing content.

Open the file "demofile2.txt" and append content to the file:

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()

#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

Open the file "demofile3.txt" and overwrite the content:

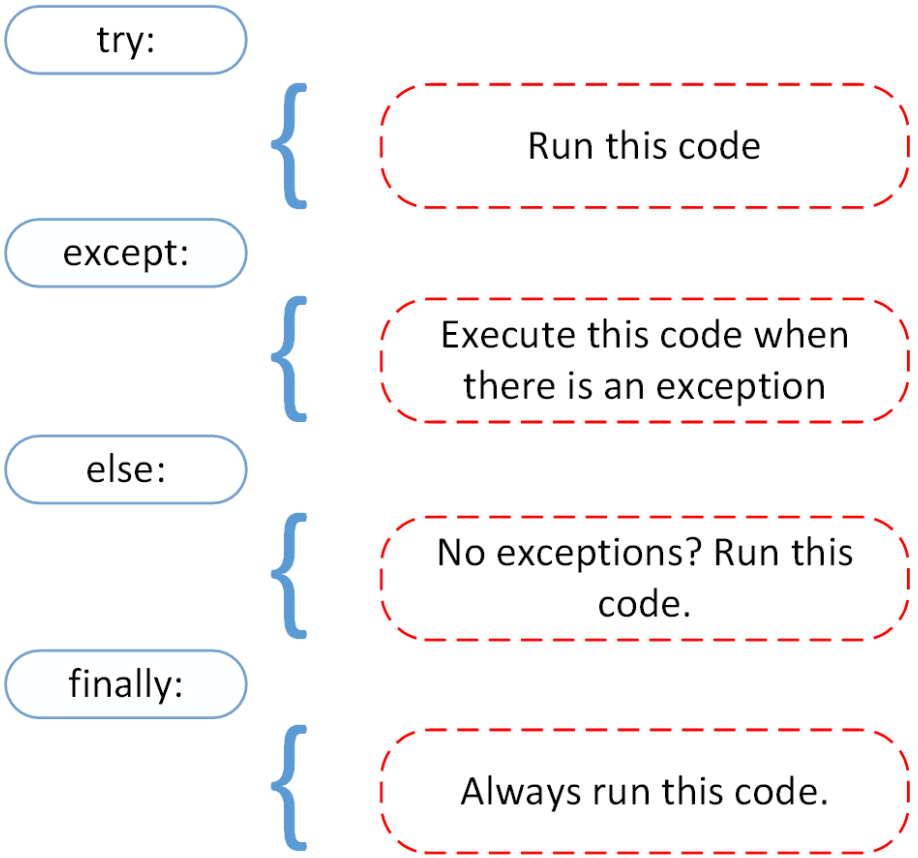
```
f = open("demofile3.txt", "w")
f.write("Woops! I have deleted the content!")
f.close()

#open and read the file after the overwriting:
f = open("demofile3.txt", "r")
print(f.read())
```

2 - Exception Handling

Python Try Except

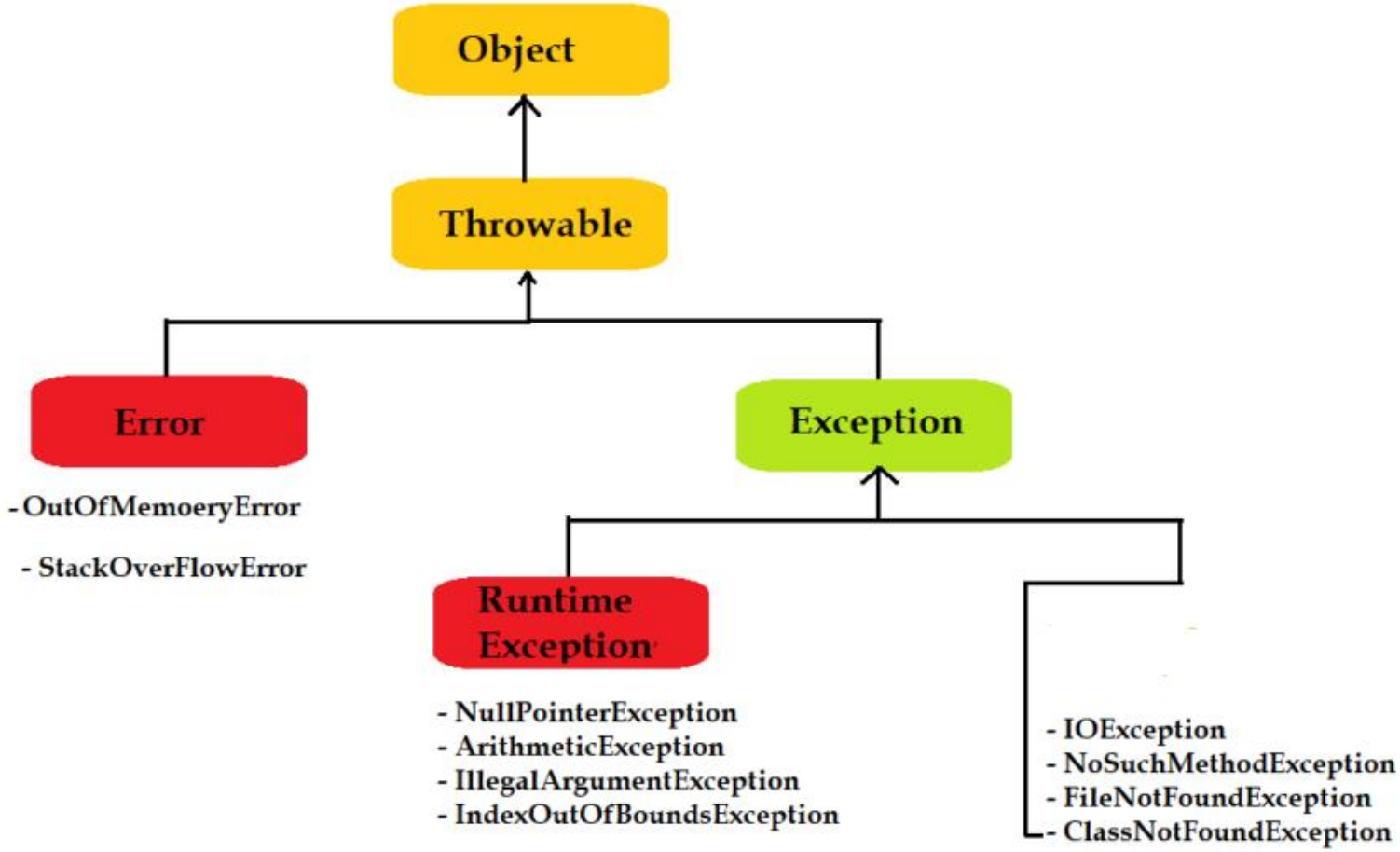
An error is a problem, bug, or human-created mistake that arises at the time of execution of a program.

- The try block lets us test a block of code for errors.
- The except block lets us handle the error.
- The else block lets us execute code when there is no error.
- The finally block lets us execute code, regardless of the result of the try- and except blocks.



Exception Class *in* Java

An **exception interrupts the flow of the program and terminates it abnormally**. The termination of the program abnormally is not recommended, and for that, we need to handle these exceptions. Java provides **Java.lang.Exception class** for handling the exceptions which inherit the properties and methods of **Object** and **Throwable class**.



Python Exception Handling

When an ERROR OCCURS, or exception as we call it, Python will normally stop and generate an error message. These exceptions can be handled using the *try statement*.

```
try:
  print(x)
except:
  print("Something went wrong")
finally:
  print("The 'try except' is finished")
```

```
try:
  f = open("demofile.txt")
  try:
    f.write("Lorum Ipsum")
  except:
    print("Something went wrong when writing to the file")
  finally:
    f.close()
except:
  print("Something went wrong when opening the file")
```

Error Types in Python

```
x = "hello"

if not type(x) is int:
  raise TypeError("Only integers are allowed")
```

```
Traceback (most recent call last):
  File "demo_ref_keyword_raise2.py", line 4, in <module>
    raise TypeError("Only integers are allowed")
TypeError: Only integers are allowed
```

We can define what kind of error to raise, and the text to print to the user.

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax obj.foo, if obj has no member named foo
EOFError	Raised if "end of file" reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by next(iterator) if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., sqrt(−5))
ZeroDivisionError	Raised when any division operator used with 0 as divisor

Exercise *for* Students

Exercise #1

Misalkan kalian sebagai seorang developer diminta untuk membuat sebuah chatbot dan harus ada information extraction spy chatbot kalian mengerti setiap informasi yang di inputkan user. Kalian diminta untuk menggunakan class modul Regex di Python untuk extract *date*, *email* dan *time* pada text di bawah ini:

Input Text:

Hallo students, email berikut (semmy@gmail.com, jian@yahoo.com, budi@start.com) adalah student yang belum tuntas class ini. Diharapkan agar dapat memasukan tugas pada 22 Juni 2024 dan 43 Agustus 2024, jam 7 AM dan 26 PM. Thank you :)

Output:

semmy@gmail.com	= Valid Email
jian@yahoo.com	= Valid Email
budi@start.com	= Invalid Email (Bukan yahoo dan gmail account)
22 Juni 2024	= Valid Date
43 Agustus 2024	= Invalid Date
7 PM	= Valid Time
26 PM	= Invalid Time

END PRESENTATION

Thank you for your attention

Instructor: S – W – T